Neural Networks & Backpropagation

Neurons, activations, the forward pass, backprop's chain rule, CNNs, and RNNs

Concept Guide 4 of 9 | Read before: PyTorch or TF/Keras lab | Companion to the Zero-to-Hero series

---

1. The Neuron: Weighted Sum + Activation

A single artificial neuron computes a weighted sum of its inputs, adds a bias, then passes the result through a nonlinear activation function.

FORMULA — A single neuron

```
output = activation(w1*x1 + w2*x2 + ... + wn*xn + b)
```

w = learned weights, b = learned bias, activation = a nonlinear function

DIAGRAM — The neuron as a diagram

```
x1 --w1--\
x2 --w2---> [SUM + bias] --> [activation] --> output
x3 --w3--/
```

MENTAL MODEL

Without an activation function, stacking layers would collapse into a single linear operation, no matter how many layers you add (a linear function of a linear function is still linear). The nonlinearity is what lets networks learn curved, complex decision boundaries.

PRACTICE THIS → **17_Neural_Networks_Deep_Dive/01_Forward_Pass**

2. Activation Functions

Different activations serve different roles: ReLU for hidden layers (fast, avoids some gradient problems), sigmoid for binary output probabilities, softmax for multi-class output probabilities.

FORMULA — Common activations

```
ReLU(x) = max(0, x)
sigmoid(x) = 1 / (1 + e^-x)
tanh(x) = (e^x - e^-x) / (e^x + e^-x)
```

ReLU is the default choice for hidden layers in modern networks

COMMON MISUNDERSTANDINGS

- Sigmoid and tanh 'saturate' for large |x| — their gradient approaches zero, which can stall learning in deep networks (the vanishing gradient problem).
- ReLU outputs exactly 0 for any negative input — a neuron that's always negative 'dies' and stops learning entirely (the dying ReLU problem), one reason variants like Leaky ReLU exist.

PRACTICE THIS → **PyTorch_Zero_to_Hero.ipynb Ch.4**

3. The Forward Pass

The forward pass computes the network's output by pushing an input through each layer in sequence — each layer's output becomes the next layer's input.

DIAGRAM — A 2-layer network's forward pass

```
input x (shape 4)
   |  Layer 1: z1 = W1 @ x + b1,  a1 = ReLU(z1)   (shape 8)
   v
a1 (shape 8)
   |  Layer 2: z2 = W2 @ a1 + b2,  output = z2     (shape 1)
   v
output (shape 1)
```

MENTAL MODEL

Each layer is just: multiply by a weight matrix, add a bias, apply an activation. Stack enough of these and the network can approximate extremely complex functions — this is the Universal Approximation idea.

PRACTICE THIS → **17_Neural_Networks_Deep_Dive/01_Forward_Pass**

4. Backpropagation: The Chain Rule at Scale

Backpropagation computes the gradient of the loss with respect to EVERY weight in the network, by applying the chain rule repeatedly, working backward from the output.

FORMULA — The core idea (2-layer case)

$$\frac{dLoss}{dW2} = \frac{dLoss}{dOutput} * \frac{dOutput}{dW2}$$

$$\frac{dLoss}{dW1} = \frac{dLoss}{dOutput} * \frac{dOutput}{da1} * \frac{da1}{dz1} * \frac{dz1}{dW1}$$

each weight's gradient is a product of local derivatives, chained back from the loss

DIAGRAM — Forward builds the graph; backward walks it in reverse

```
 forward:   x --W1--> z1 --ReLU--> a1 --W2--> z2 = output --> loss
 backward:  dL/dx <-- dL/dW1 <-- dL/da1 <-- dL/dz1 <-- dL/dW2 <-- dL/doutput <--
dL/dL=1
```

MENTAL MODEL

Backpropagation is 'the chain rule, applied layer by layer, walking backward through the network.' Each layer only needs to know how to compute ITS OWN local derivative and pass the accumulated gradient further back — no layer needs global knowledge of the whole network.

COMMON MISUNDERSTANDINGS

- In very deep networks, repeatedly multiplying small gradients (from saturating activations like sigmoid) can shrink the gradient toward zero by the time it reaches early layers — the vanishing gradient problem, one motivation for ReLU and for residual/skip connections (used heavily in Transformers).
- Gradients ACCUMULATE by default in PyTorch — you must zero them before each new backward pass, or they silently corrupt training.

PRACTICE THIS → **17_Neural_Networks_Deep_Dive/02_Backpropagation; PyTorch_Zero_to_Hero.ipynb Ch.3**

5. CNNs: Convolution as a Sliding Dot Product

A convolutional layer slides a small filter (kernel) across an input, computing a dot product at each position — the same filter is reused everywhere (parameter sharing), which is why CNNs are efficient for images.

DIAGRAM — A 3x3 filter sliding over an image

```
image:  [ a b c d ]        filter:  [ w1 w2 ]
        [ e f g h ]                 [ w3 w4 ]
        [ i j k l ]

output[0][0] = a*w1 + b*w2 + e*w3 + f*w4   (top-left 2x2 patch)
output[0][1] = b*w1 + c*w2 + f*w3 + g*w4   (slide right by 1)
... slide the SAME filter across the whole image
```

MENTAL MODEL

Because the filter is reused at every position, a CNN learns 'detect this pattern (like an edge or a texture) wherever it appears in the image' — instead of learning a separate detector for every possible pixel location.

PRACTICE THIS → **17_Neural_Networks_Deep_Dive/03_CNN**

6. RNNs: Recurrence and the Vanishing Gradient Over Time

A recurrent network processes a sequence one step at a time, carrying a 'hidden state' forward as memory of everything seen so far.

FORMULA — The recurrence relation

```
h_t = activation(W_h * h_(t-1) + W_x * x_t + b)
```

the new hidden state depends on the previous hidden state AND the current input

COMMON MISUNDERSTANDINGS

- Backpropagating through many time steps means multiplying many gradients together — exactly the vanishing (or exploding) gradient problem from Section 4, but now across TIME instead of across LAYERS.
- This is precisely the motivation for LSTMs/GRUs (gating mechanisms that let gradients flow more directly) and, ultimately, for attention/Transformers abandoning recurrence entirely.

PRACTICE THIS → **17_Neural_Networks_Deep_Dive/04_RNN**